

ACM Lifecycle and State Propagation Model v0.1

Agentic Configuration Management — Lifecycle and State Propagation Model

Statut : Working Draft

Version : 0.1.0

Spécification parente : ACM Core Specification v0.1

Portée : lifecycle, qualité, assurance, impact, éligibilité et propagation d'état

Implémentation de référence envisagée : Python 3.11+, Pydantic, JSON/JSONL

Langue normative : anglais technique pour les noms d'objets, d'états et de propriétés ; français pour les explications

1. Objet

Le présent document définit le modèle normatif de cycle de vie et de propagation d'état d'Agentic Configuration Management.

Il complète ACM Core Specification v0.1 en précisant :

1. les machines à états applicables aux objets de configuration, aux baselines, aux instances runtime et aux objets créés dynamiquement ;
2. les transitions autorisées et interdites ;
3. les événements et conditions déclenchant ces transitions ;
4. les relations entre lifecycle, qualité et assurance ;
5. les états calculés d'impact et d'éligibilité ;
6. les règles de propagation au sein du graphe de configuration ;
7. les règles de validité et de péremption des preuves ;
8. les règles applicables aux agents et objets créés au runtime ;
9. les invariants devant être vérifiés par une implémentation conforme ;
10. les scénarios minimaux de validation du modèle.

Le présent document ne définit pas :

- un moteur général de workflow ;
- un langage complet de règles ;
- une plateforme de gouvernance organisationnelle ;
- un système de déploiement ;
- un moteur d'évaluation des LLM ;
- une taxonomie exhaustive de qualité ;
- une logique probabiliste d'agrégation ;
- un mécanisme distribué de consensus sur les états.

Le modèle doit rester déterministe, explicable et implémentable localement.

2. Vocabulaire normatif

Les termes suivants sont utilisés au sens normatif :

- **MUST / DOIT** : exigence obligatoire ;
- **MUST NOT / NE DOIT PAS** : interdiction ;
- **SHOULD / DEVRAIT** : recommandation forte ;
- **SHOULD NOT / NE DEVRAIT PAS** : pratique déconseillée ;
- **MAY / PEUT** : fonctionnalité optionnelle.

Une implémentation est dite conforme à **ACM Lifecycle and State Propagation Model v0.1** si elle respecte toutes les exigences marquées **MUST**.

3. Principes fondamentaux

3.1. Séparation des dimensions

ACM distingue cinq dimensions d'état :

1. `lifecycle_state` ;
2. `quality_state` ;
3. `assurance_state` ;
4. `impact_state` ;
5. `eligibility_state` .

Ces dimensions **NE DOIVENT PAS** être fusionnées dans un état global unique.

En particulier :

- le lifecycle ne représente pas la qualité ;
 - la qualité ne représente pas la quantité de preuve ;
 - l'assurance ne représente pas la réussite des évaluations ;
 - l'impact ne signifie pas qu'un objet est invalide ;
 - l'éligibilité ne modifie pas le jugement intrinsèque porté sur l'objet.
-

3.2. États intrinsèques et états calculés

Un ACI possède deux catégories d'état.

États intrinsèques

Ils sont attachés à une révision exacte :

```
lifecycle_state
quality_state
assurance_state
```

Ils résultent d'une création, d'une décision, d'une évaluation ou d'une transition explicitement enregistrée.

États calculés

Ils sont dérivés du graphe, des dépendances et des preuves :

```
impact_state  
eligibility_state  
effective_quality  
effective_assurance
```

Les états calculés DOIVENT pouvoir être recalculés sans modifier le contenu intrinsèque de l'ACI.

3.3. Absence de propagation directe du lifecycle

Le lifecycle d'une dépendance NE DOIT PAS être copié sur son parent.

Par exemple :

```
PromptDefinition.lifecycle_state = deprecated
```

n'implique pas :

```
AgentDefinition.lifecycle_state = deprecated
```

Il peut en revanche produire :

```
AgentDefinition.impact_state = impacted  
AgentDefinition.eligibility_state = warning
```

Le lifecycle reste une propriété de la révision concernée.

3.4. Propagation restrictive plutôt que destructive

Une dépendance problématique peut empêcher la promotion d'un composite sans modifier sa qualité intrinsèque.

Exemple :

```
ToolDefinition.effective_quality = nok
```

peut produire :

```
AgentDefinition.eligibility_state = blocked
```

mais NE DOIT PAS automatiquement produire :

```
AgentDefinition.quality_state = nok
```

Cette distinction préserve la provenance des jugements et évite les propagations irréversibles.

3.5. Validation liée à une révision exacte

Toute validation, approbation ou preuve normative DOIT cibler un `revision_id`, un digest ou un ensemble exact de révisions.

Une validation portant uniquement sur l'identité logique d'un objet est insuffisante.

```
aci:prompt:planner-system
```

ne suffit pas.

Il faut identifier :

```
aci:prompt:planner-system  
revision_id = 01J...  
digest = sha256:...
```

3.6. Recalcul déterministe

Pour une même configuration, un même ensemble de preuves et une même politique de propagation, le calcul des états effectifs DOIT produire le même résultat.

$$compute(C, E, P) = compute(C, E, P)$$

où :

- C est le graphe de configuration ;
 - E est l'ensemble des preuves ;
 - P est la politique de propagation.
-

4. Modèle général d'état

4.1. Structure normative

Chaque ACI DEVRAIT exposer une vue d'état équivalente à :

```
{
  "declared_status": {
    "lifecycle_state": "validated",
    "quality_state": "ok",
    "assurance_state": "assessed"
  },
  "computed_status": {
    "impact_state": "current",
    "eligibility_state": "eligible",
    "effective_quality": "ok",
    "effective_assurance": "assessed",
    "reasons": []
  }
}
```

La vue `computed_status` PEUT être stockée en cache, mais elle DOIT être considérée comme dérivée.

4.2. Reasons

Chaque état calculé différent de sa valeur nominale DOIT être accompagné d'au moins une raison structurée.

Exemple :

```
{
  "code": "ACM-PROP-QUALITY-BLOCK",
  "source_ref": {
    "id": "aci:tool:web-search",
    "revision_id": "01J..."
  },
  "relation_id": "rel:agent:uses-tool:web-search",
  "message": "A required dependency has effective quality NOK."
}
```

Une raison DEVRAIT contenir :

- un code stable ;
- la source du changement ;
- la relation empruntée ;
- l'état observé ;

- la règle appliquée ;
- le niveau de sévérité.

5. Machine à états des révisions d'ACI

5.1. États normatifs

Le lifecycle d'une révision d'ACI utilise :

```
draft
candidate
validated
deprecated
archived
```

Ces états remplacent, pour cette sous-spécification, la séquence antérieure :

```
draft
experimental
testing
validated
production
deprecated
archived
```

Les notions `experimental`, `testing` et `production` NE SONT PAS des lifecycle states normatifs dans ce modèle.

Elles peuvent être représentées séparément par :

```
validation_stage
deployment_environment
deployment_state
```

Cette séparation évite de confondre maturité de configuration et déploiement effectif.

5.2. Sémantique des états

`draft`

Révision en cours de création ou de modification.

Propriétés :

- son contenu peut évoluer ;
- elle n'est pas éligible à une baseline released ;
- son assurance est généralement `unassessed` ;
- sa qualité peut être `unknown`, `to_improve`, `ok` ou `nok`.

`candidate`

Révision proposée pour validation.

Propriétés :

- son contenu DOIT être figé pendant la validation ;
- son digest DOIT être calculé ;
- les références obligatoires DOIVENT être résolubles ;
- les évaluations requises peuvent être en cours ;
- elle n'est pas encore éligible à une baseline released.

`validated`

Révision dont les conditions de validation applicables sont satisfaites.

Propriétés :

- son contenu est immuable ;
- son digest est vérifié ;
- son assurance effective DOIT être `assessed` ;
- sa qualité effective DOIT être `ok` ou `to_improve` ;
- elle peut être incluse dans une baseline candidate ou released, sous réserve des dépendances.

`deprecated`

Révision encore identifiable et potentiellement utilisable, mais dont l'usage futur est déconseillé.

Propriétés :

- les baselines existantes peuvent continuer à la référencer ;
- une nouvelle baseline released NE DEVRAIT PAS l'inclure ;
- son utilisation produit au minimum un warning ;
- elle peut avoir un successeur déclaré par `replaces`.

`archived`

Révision retirée de l'usage actif.

Propriétés :

- elle reste consultable à des fins d'audit ;
- elle ne peut plus être introduite dans une nouvelle baseline ;
- elle ne peut plus être directement restaurée vers un état actif ;

- toute réutilisation nécessite la création d'une nouvelle révision.

5.3. Matrice de transitions

État courant	État cible	Autorisée	Événement normatif
draft	candidate	Oui	aci.submitted
draft	archived	Oui	aci.abandoned
candidate	draft	Oui	aci.rework_requested
candidate	validated	Oui	aci.validation_approved
candidate	archived	Oui	aci.rejected
validated	deprecated	Oui	aci.deprecated
validated	archived	Non directe	passer par deprecated
deprecated	archived	Oui	aci.archived
deprecated	validated	Non	nouvelle révision requise
archived	tout autre état	Non	nouvelle révision requise
candidate	deprecated	Non	révision non encore validée
draft	validated	Non	validation candidate requise

5.4. Conditions de transition

draft → candidate

La transition DOIT satisfaire :

```
digest_present = true
schema_valid = true
required_references_resolvable = true
content_frozen = true
```

Les dépendances ne sont pas obligatoirement validées à ce stade, mais leur état DOIT être connu.

candidate → validated

La transition DOIT satisfaire :

```
effective_assurance = assessed
effective_quality ∈ {ok, to_improve}
eligibility_state ≠ blocked
```



```
all_required_checks_completed = true
all_blocking_checks_passed = true
required_approval_present = true
```

Une politique de projet PEUT exiger :

```
effective_quality = ok
```

candidate → draft

Cette transition DOIT être déclenchée lorsqu'au moins une condition de validation nécessite une modification du contenu.

Toute modification crée un nouveau digest et PEUT créer un nouveau `revision_id` selon le modèle de stockage retenu.

validated → deprecated

Cette transition PEUT être déclenchée par :

- l'existence d'une révision de remplacement ;
- une vulnérabilité ;
- une baisse de performance ;
- une incompatibilité ;
- une décision de maintenance ;
- l'obsolescence d'une dépendance ;
- la disparition d'un fournisseur ou d'un modèle.

Elle ne modifie pas les baselines historiques.

deprecated → archived

La transition DEVRAIT exiger :

- l'absence d'usage actif connu ;
- l'existence d'un remplacement ou d'une justification ;
- une décision explicite d'archivage.

5.5. Acteurs de transition

Une transition DOIT enregistrer un `actor`.

Types d'acteurs autorisés :

```
user
reviewer
maintainer
```

```
pipeline
acm_engine
external_system
agent
```

Une transition vers `validated`, `released` ou `registered` NE DOIT PAS être effectuée par un agent runtime sans approbation explicitement autorisée par une `PolicyDefinition`.

Par défaut :

```
agent MAY propose
agent MUST NOT approve
```

5.6. Événement de transition

```
{
  "event_type": "aci.validation_approved",
  "event_id": "evt_01J...",
  "target_ref": {
    "id": "aci:agent:planner",
    "revision_id": "01J...",
    "digest": "sha256:..."
  },
  "previous_state": "candidate",
  "new_state": "validated",
  "actor": {
    "type": "reviewer",
    "id": "user:audrey"
  },
  "evidence_refs": [
    "evidence:eval:planner-regression:001"
  ],
  "timestamp": "2026-07-27T12:00:00Z",
  "reason": "All required checks passed."
}
```

6. Machine à états des baselines

6.1. États normatifs

```
candidate
released
```

superseded
withdrawn

6.2. Sémantique

candidate

Snapshot exact en cours de vérification.

Une baseline candidate :

- possède un digest ;
- référence des révisions exactes ;
- n'est pas encore autorisée comme référence de release ;
- peut être remplacée par une nouvelle candidate.

released

Baseline approuvée, immuable et utilisable comme référence d'exécution gouvernée.

superseded

Baseline remplacée par une baseline released plus récente.

Elle reste valide historiquement et peut encore être utilisée si une politique l'autorise.

withdrawn

Baseline explicitement retirée en raison d'un problème significatif.

Son utilisation future est interdite sauf procédure exceptionnelle d'audit ou de reproduction.

6.3. Matrice de transitions

État courant	État cible	Autorisée	Événement
candidate	released	Oui	baseline.released
candidate	withdrawn	Oui	baseline.rejected
released	superseded	Oui	baseline.superseded
released	withdrawn	Oui	baseline.withdrawn
superseded	withdrawn	Oui	baseline.withdrawn
withdrawn	released	Non	nouvelle baseline requise
superseded	released	Non	nouvelle baseline requise

6.4. Conditions de release

Une baseline candidate ne peut devenir `released` que si :

```
baseline_digest_valid = true
all_exact_references_resolved = true
all_required_items.lifecycle_state = validated
all_required_items.eligibility_state = eligible
all_required_items.effective_assurance = assessed
all_required_items.effective_quality ∈ {ok, to_improve}
no_required_item.lifecycle_state ∈ {draft, candidate, archived}
no_required_item.eligibility_state = blocked
validation_report.valid = true
approval_present = true
```

Par défaut, la présence d'un ACI `deprecated` dans une nouvelle baseline produit :

```
eligibility_state = blocked
```

Une politique explicite PEUT transformer ce blocage en warning avec dérogation documentée.

6.5. Baselines historiques

Une baseline déjà released NE DEVIENT PAS automatiquement invalide lorsqu'un de ses ACI est ultérieurement marqué `deprecated`.

Elle peut recevoir un statut opérationnel externe :

```
attention_required
reassessment_required
withdrawal_recommended
```

La baseline ne change d'état que par événement explicite.

7. Machine à états des instances runtime

7.1. États normatifs

```
created
ready
running
waiting
completed
```

```
failed
cancelled
terminated
```

7.2. Matrice minimale

État courant	États cibles autorisés
created	ready, failed, cancelled
ready	running, cancelled, failed
running	waiting, completed, failed, cancelled
waiting	running, failed, cancelled
completed	terminated
failed	terminated
cancelled	terminated
terminated	aucun

Une transition non présente dans cette table DOIT être rejetée en mode strict.

7.3. États terminaux

```
completed
failed
cancelled
terminated
```

`terminated` représente la fin de l'existence active de l'instance.

Une instance `completed`, `failed` ou `cancelled` PEUT rester inspectable avant l'émission de `node.terminated`.

7.4. Runtime et lifecycle de configuration

Le runtime state d'une instance NE DOIT PAS modifier le lifecycle de sa définition.

Exemple :

```
RuntimeAgentInstance.state = failed
```

n'implique pas :

```
AgentDefinition.quality_state = nok
```

Il peut produire une nouvelle preuve ou un signal d'impact nécessitant une réévaluation.

8. Machine de promotion des objets créés au runtime

8.1. Portée

Cette machine s'applique aux objets générés pendant une exécution et susceptibles d'être conservés :

- agents dynamiques ;
 - prompts générés ;
 - workflows dérivés ;
 - politiques proposées ;
 - configurations résolues.
-

8.2. États normatifs

```
ephemeral  
retained  
candidate  
registered  
rejected  
expired
```

8.3. Sémantique

`ephemeral`

Objet limité à l'exécution en cours.

Il est enregistré dans la provenance mais n'est pas réutilisable comme ACI permanent.

`retained`

Objet conservé pour inspection, comparaison ou analyse.

Il n'est pas encore proposé pour réutilisation.

`candidate`

Objet extrait et normalisé en vue d'une éventuelle inscription au registre ACM.

`registered`

Objet devenu une nouvelle révision d'ACI après validation et approbation.

`rejected`

Objet examiné mais refusé.

`expired`

Objet dont la durée de conservation ou d'applicabilité a pris fin sans promotion.

8.4. Transitions

```
ephemeral → retained  
ephemeral → expired  
retained → candidate  
retained → rejected  
retained → expired  
candidate → registered  
candidate → rejected  
candidate → retained
```

Les transitions suivantes sont interdites :

```
ephemeral → registered  
rejected → registered  
expired → registered
```

Un objet rejeté ou expiré peut seulement servir de source à une nouvelle révision explicitement créée.

8.5. Conditions de promotion

La transition `candidate → registered` DOIT exiger :

- une normalisation dans un type ACI reconnu ;
- un nouvel `revision_id` ;
- un digest valide ;
- une provenance complète ;
- un lifecycle initial `draft` ou `candidate` ;
- les évaluations requises ;
- une approbation humaine ou une politique explicite ;
- l'absence d'escalade de permissions ;
- l'absence de secrets non autorisés.

Un objet runtime NE DOIT PAS être enregistré directement comme `validated`.

9. Quality State

9.1. États normatifs

ok
to_improve
nok
unknown

9.2. Sémantique

ok

Les critères de qualité applicables ont réussi selon les seuils définis.

to_improve

L'objet demeure utilisable selon la politique applicable, mais au moins un critère non bloquant n'atteint pas la cible recommandée.

nok

Au moins un critère bloquant a échoué ou une décision explicite conclut que l'objet est inacceptable.

unknown

Aucun jugement de qualité suffisant n'est disponible pour cette révision et ce périmètre.

9.3. Qualité et assurance

Les combinaisons suivantes sont valides :

Quality	Assurance	Interprétation
ok	assessed	qualité positive soutenue par les preuves requises
ok	unassessed	appréciation positive non suffisamment démontrée
nok	assessed	évaluation complète ayant révélé un échec
unknown	assessed	évaluations complètes mais résultats non concluants
to_improve	partially_assessed	amélioration nécessaire, preuves incomplètes

assessed NE SIGNIFIE PAS passed .

9.4. Qualité scalaire et dimensions

ACM Core v0.1 utilise un état global scalaire.

Cependant, l'implémentation PEUT conserver des dimensions détaillées :

```
{
  "functional": "ok",
  "security": "nok",
  "cost": "to_improve",
  "latency": "ok",
  "robustness": "unknown"
}
```

L'état global DOIT alors être calculé selon une politique documentée.

La politique par défaut est :

1. tout échec bloquant produit `nok` ;
2. sinon toute insuffisance non bloquante produit `to_improve` ;
3. sinon toute dimension inconnue produit `unknown` ;
4. sinon l'état est `ok`.

10. Assurance State

10.1. États normatifs

```
unassessed
partially_assessed
assessed
```

10.2. Définition par couverture

Soit :

- $R(x)$ l'ensemble des contrôles requis et applicables à x ;
- $E(x)$ l'ensemble des contrôles requis disposant d'une preuve valide.

La couverture est :

$$coverage(x) = \begin{cases} 1 & \text{si } |R(x)| = 0 \\ \frac{|E(x)|}{|R(x)|} & \text{sinon} \end{cases}$$

L'état d'assurance est :

$$A(x) = \begin{cases} unassessed & coverage(x) = 0 \\ partially_assessed & 0 < coverage(x) < 1 \\ assessed & coverage(x) = 1 \end{cases}$$

Cette couverture mesure la complétude des preuves, non leur réussite.

10.3. Preuve applicable

Une preuve est applicable à un ACI si toutes les conditions suivantes sont satisfaites :

```
target revision matches
target digest matches
evidence scope includes target
environment constraints match
model constraints match
dependency constraints match
evidence is not expired
evidence is not invalidated
```

10.4. Structure minimale d'une preuve

```
{
  "evidence_id": "evidence:eval:planner:001",
  "evidence_type": "evaluation",
  "target": {
    "type": "aci_revision",
    "id": "aci:agent:planner",
    "revision_id": "01J...",
    "digest": "sha256:..."
  },
  "scope": {
    "environment": "local",
    "scenario": "planning-regression",
    "dimensions": ["functional", "robustness"]
  },
  "result": "pass",
  "blocking": true,
  "produced_at": "2026-07-27T10:00:00Z",
  "valid_until": null,
  "dependency_snapshot": [
    {
      "id": "aci:prompt:planner-system",
      "revision_id": "01J..."
    }
  ],
  "producer": {
    "type": "pipeline",
    "id": "acm-eval"
  }
}
```

10.5. Non-transitivité par défaut

Les preuves NE SONT PAS transitives par défaut.

Une preuve visant un workflow ne prouve pas automatiquement la qualité individuelle de tous ses agents.

Une preuve visant un agent ne prouve pas automatiquement le workflow complet.

Une preuve peut couvrir une composition uniquement si son `scope` identifie explicitement cette composition et les révisions concernées.

11. Impact State

11.1. États normatifs

`current`
`impacted`
`stale`

11.2. Sémantique

`current`

Aucun changement connu de dépendance ou de condition applicable n'exige une nouvelle analyse.

`impacted`

Un changement pertinent est détecté, mais ses conséquences n'ont pas encore été complètement analysées.

`stale`

Au moins une preuve, validation ou décision précédemment applicable ne peut plus être considérée comme actuelle.

11.3. Ordre de sévérité

`current < impacted < stale`

L'agrégation utilise l'état le plus sévère.

11.4. Déclencheurs

Un ACI devient au minimum `impacted` lorsque :

- une dépendance obligatoire change de révision ;
- une politique applicable change ;
- un modèle ou un outil est déprécié ;
- une relation structurelle est ajoutée ou supprimée ;
- une vulnérabilité affecte une dépendance ;
- un résultat runtime significatif remet en cause une hypothèse.

Il devient `stale` lorsque :

- une preuve dépend d'une révision remplacée ;
- un digest couvert par la preuve ne correspond plus ;
- une condition d'environnement n'est plus satisfaite ;
- la date de validité d'une preuve est dépassée ;
- une dépendance snapshotée a changé ;
- une baseline a été retirée ;
- une évaluation requise devient non applicable ou non reproductible.

11.5. Réversibilité

`impacted` ou `stale` peut redevenir `current` uniquement après :

- analyse d'impact ;
- recalcul des dépendances ;
- renouvellement ou confirmation des preuves ;
- enregistrement d'un événement de résolution.

La disparition de la cause technique ne suffit pas à restaurer automatiquement `current` .

12. Eligibility State

12.1. États normatifs

`eligible`
`warning`
`blocked`

12.2. Sémantique

`eligible`

L'objet peut participer à l'opération visée, notamment validation, release ou promotion.

warning

L'opération reste possible, mais nécessite une justification, une dérogation ou une attention particulière.

blocked

L'opération est interdite tant que la cause du blocage n'est pas corrigée ou explicitement levée par une politique autorisée.

12.3. Éligibilité contextuelle

L'éligibilité dépend d'une opération.

Une implémentation DEVRAIT pouvoir représenter :

```
validation_eligibility
release_eligibility
runtime_eligibility
promotion_eligibility
```

ACM Core v0.1 PEUT utiliser un seul `eligibility_state` lorsqu'une seule opération est évaluée à la fois.

Le rapport DOIT alors indiquer le contexte :

```
{
  "eligibility_context": "baseline_release",
  "eligibility_state": "blocked"
}
```

13. Relations et politiques de propagation

13.1. Relations concernées

Les règles de propagation s'appliquent principalement aux relations du `ConfigurationGraph` :

```
contains
uses_prompt
uses_tool
uses_model
governed_by
evaluated_under
```

```
can_instantiate
templates
```

13.2. Attributs de propagation

Chaque relation DEVRAIT contenir :

```
{
  "required": true,
  "propagation_policy": "blocking",
  "assurance_dependency": true,
  "impact_dependency": true
}
```

13.3. required

```
true
false
```

Une dépendance requise participe aux calculs bloquants.

Une dépendance optionnelle ne bloque pas automatiquement son parent.

13.4. propagation_policy

Valeurs normatives :

```
blocking
warning
informational
none
```

blocking

Une défaillance de la cible peut bloquer l'éligibilité de la source.

warning

Une défaillance de la cible produit au maximum un warning.

informational

La relation est rapportée mais n'affecte pas l'éligibilité.

none

Aucune propagation automatique.

13.5. Valeurs par défaut

Relation	Required par défaut	Propagation par défaut
<code>contains</code> système → workflow	Oui	<code>blocking</code>
<code>contains</code> workflow → agent	Oui	<code>blocking</code>
<code>uses_prompt</code>	Oui	<code>blocking</code>
<code>uses_tool</code>	Oui	<code>blocking</code>
<code>uses_model</code>	Oui si présent	<code>blocking</code>
<code>governed_by</code>	Oui	<code>blocking</code>
<code>evaluated_under</code>	Non	<code>warning</code>
<code>can_instantiate</code>	Non	<code>warning</code>
<code>templates</code>	Oui	<code>blocking</code>

Une implémentation PEUT modifier ces valeurs par politique explicite.

14. Types de propagation

ACM distingue trois propagations normatives.

14.1. Propagation d'impact

Elle répond à :

Quels objets peuvent être affectés par une modification ?

Elle suit les relations où :

```
impact_dependency = true
```

Si une cible devient `impacted`, la source devient au minimum `impacted`.

Si une cible devient `stale` et que la source possède une preuve dépendant de cette cible, la source devient `stale`.

14.2. Propagation d'éligibilité

Elle répond à :

Un objet composite peut-il être validé, promu ou inclus dans une baseline ?

Elle utilise :

- la qualité effective ;
- l'assurance effective ;
- le lifecycle ;
- la politique de relation ;
- le caractère obligatoire de la dépendance.

14.3. Propagation d'assurance

Elle répond à :

Les preuves disponibles couvrent-elles encore la composition actuelle ?

Une modification de dépendance ne change pas directement le `assurance_state` déclaré.

Elle modifie l'applicabilité des preuves, puis le moteur recalcule :

effective_assurance

15. Calcul de la qualité effective

15.1. Qualité déclarée

Soit :

$$Q_d(x)$$

la qualité déclarée de l'ACI x .

15.2. Qualité calculée depuis les dépendances

Soit `Req(x)` l'ensemble des dépendances requises avec une politique `blocking`.

$$Q_c(x) = \begin{cases} nok & \exists d \in Req(x) : Q_e(d) = nok \text{ to_improve} \neg \exists d : Q_e(d) = nok \wedge \exists d : Q_e(d) = to_improve \text{ unknown} \end{cases}$$

15.3. Qualité effective

Ordre de sévérité :

`ok < unknown < to_improve < nok`

Pour v0.1 :

$$Q_e(x) = \text{worst}(Q_d(x), Q_c(x))$$

Cette règle est une projection opérationnelle simplifiée.

Elle NE DOIT PAS être interprétée comme une vérité sémantique générale sur la qualité.

15.4. Dépendances non bloquantes

Une dépendance `warning` avec qualité `nok` :

- NE DOIT PAS rendre `effective_quality = nok` ;
 - DOIT contribuer à `eligibility_state = warning` ;
 - DOIT apparaître dans `reasons`.
-

16. Calcul de l'assurance effective

16.1. Assurance intrinsèque

L'assurance intrinsèque est calculée depuis les preuves directement attachées à la révision.

$$A_d(x) = \text{coverageState}(x)$$

16.2. Assurance composite

Une composition est `assessed` uniquement si :

1. toutes ses preuves directes requises sont applicables ;
2. toutes les dépendances marquées `assurance_dependency = true` sont effectivement `assessed` ;
3. aucune preuve requise n'est `stale`.

$$A_e(x) = \begin{cases} \text{unassessed} & A_d(x) = \text{unassessed} \wedge \text{aucune couverture applicable} \\ \text{partially_assessed} & A_d(x) / \end{cases}$$

Une politique PEUT autoriser une preuve de composition à couvrir explicitement certaines dépendances et à éviter une agrégation complète.

Cette dérogation DOIT être visible dans le `scope` de la preuve.

17. Calcul de l'éligibilité

17.1. Éligibilité d'une révision candidate à la validation

Une révision candidate est `blocked` si :

- son schéma est invalide ;
- une référence obligatoire est manquante ;
- son `effective_quality = nok` ;
- son `effective_assurance = unassessed` pour une validation finale ;
- une dépendance obligatoire est `draft`, `candidate` ou `archived` ;
- une relation blocking produit une violation ;
- son digest est invalide.

Elle est `warning` si :

- son `effective_quality = to_improve` ;
- une dépendance est `deprecated` ;
- une relation warning est problématique ;
- une preuve non bloquante est manquante.

Sinon elle est `eligible`.

17.2. Éligibilité à une baseline released

Une révision est `blocked` pour une nouvelle baseline si :

```
lifecycle_state ≠ validated
effective_quality = nok
effective_assurance ≠ assessed
impact_state = stale
required dependency eligibility = blocked
digest invalid
```

Elle est `warning` si :

```
effective_quality = to_improve
impact_state = impacted
optional dependency = nok
non-blocking evidence missing
```

Sinon elle est `eligible`.

17.3. Agrégation

Ordre de sévérité :

```
eligible < warning < blocked
```

Pour un composite :

$$Eligibility(x) = \max(Eligibility_{self}(x), Eligibility(d_1), \dots, Eligibility(d_n))$$

La propagation d'un enfant ne s'applique que selon la politique portée par la relation.

18. Invalidation et péremption des preuves

18.1. Causes normatives d'invalidation

Une preuve DOIT devenir non applicable lorsqu'au moins une condition est vraie :

1. le `revision_id` cible ne correspond plus ;
 2. le digest cible ne correspond plus ;
 3. une dépendance snapshotée a changé ;
 4. un paramètre couvert a changé ;
 5. l'environnement ne correspond plus au scope ;
 6. la preuve a dépassé `valid_until` ;
 7. le producteur de preuve est révoqué ;
 8. la preuve est explicitement annulée ;
 9. un incident invalide son hypothèse ;
 10. la politique d'évaluation applicable a changé de manière incompatible.
-

18.2. Types d'invalidation

```
target_changed  
dependency_changed  
environment_changed  
expired  
revoked  
policy_changed  
incident_invalidated  
manual
```

18.3. Effets

L'invalidation d'une preuve :

- NE MODIFIE PAS son contenu historique ;
 - marque son applicabilité comme `stale` ou `invalid` ;
 - déclenche le recalcul de l'assurance ;
 - déclenche la propagation d'impact vers les dépendants ;
 - peut bloquer une nouvelle release.
-

18.4. Preuves historiques

Une preuve non applicable à la configuration actuelle reste valide comme témoignage historique de la configuration qu'elle couvrait.

Elle NE DOIT PAS être supprimée.

19. Nouvelle révision et héritage d'état

19.1. Règle par défaut

Toute modification du contenu canonique d'un ACI produit :

```
new revision_id
new digest
lifecycle_state = draft
quality_state = unknown
assurance_state = unassessed
impact_state = current
eligibility_state = blocked
```

`impact_state = current` signifie ici que la nouvelle révision n'est pas affectée par un changement externe connu ; cela ne signifie pas qu'elle est validée.

19.2. Réutilisation de preuves

Une preuve d'une révision précédente NE DOIT PAS être automatiquement attachée à la nouvelle révision.

Elle PEUT être réutilisée si un mécanisme explicite démontre que :

- les propriétés couvertes n'ont pas changé ;
- les dépendances couvertes sont identiques ;
- la preuve est encore temporellement valide ;
- une politique autorise cette réutilisation.

La réutilisation DOIT produire une nouvelle relation de provenance ou une attestation.

19.3. Changements non substantiels

Une implémentation PEUT distinguer :

```
metadata_only
non_behavioral
behavioral
interface_breaking
security_relevant
```

Cependant, tout changement modifiant le digest crée une nouvelle révision.

La classification peut seulement influencer les évaluations à relancer.

20. Agents dynamiques et propagation

20.1. État initial

Une instance dynamique conforme à une factory validée DOIT commencer avec :

```
runtime_state = created
runtime_promotion_state = ephemeral
quality_state = unknown
```

Son assurance initiale dépend de sa configuration résolue.

20.2. Instance sans override comportemental

Si toutes les conditions suivantes sont satisfaites :

- la factory est `validated` ;
- le template est `validated` ;
- tous les outils et modèles sont ceux de la baseline ;
- aucun prompt comportemental n'est remplacé ;
- les permissions respectent le plafond ;
- les paramètres résolus restent dans les plages validées ;

alors l'instance PEUT recevoir :

```
effective_assurance = partially_assessed
```

Elle NE DOIT PAS recevoir automatiquement `assessed`, car les preuves du template ne couvrent pas nécessairement le contexte runtime.

20.3. Instance avec override comportemental

Les overrides suivants sont considérés comme comportementaux par défaut :

```
prompt content
system instructions
model identity
tool set
permissions
delegation policy
termination rules
memory source
retrieval source
```

Une instance comportant un tel override commence avec :

```
quality_state = unknown
effective_assurance = unassessed
```

jusqu'à l'enregistrement des contrôles runtime requis.

20.4. Propagation vers le graphe runtime

Une instance dynamique `unknown` ou `unassessed` NE REND PAS automatiquement le graphe runtime `nok`.

Elle produit :

```
runtime_graph.effective_assurance ≤ partially_assessed
```

et éventuellement :

```
runtime_graph.eligibility_state = warning
```

Une instance dynamique devient bloquante si :

- elle n'est pas traçable ;
- elle dépasse les permissions autorisées ;
- elle n'est pas créée par une factory valide ;
- elle utilise une définition ou un outil non autorisé ;
- une évaluation bloquante échoue.

20.5. Promotion vers un ACI

La performance observée au runtime PEUT justifier la conservation d'un agent, mais ne constitue pas une validation suffisante.

La promotion requiert toujours le lifecycle :

```
ephemeral
→ retained
→ candidate
→ registered as draft ACI
→ candidate ACI
→ validated ACI
```

21. Algorithme de propagation

21.1. Préconditions

Le graphe de configuration DOIT être résolu.

Pour chaque relation, les éléments suivants doivent être connus :

- source ;
- cible ;
- type ;
- caractère requis ;
- politique de propagation ;
- dépendance d'assurance ;
- dépendance d'impact.

21.2. Ordre de calcul

Le moteur DOIT calculer dans l'ordre :

1. validité structurelle ;
2. applicabilité des preuves ;
3. assurance intrinsèque ;
4. impact local ;
5. qualité calculée ;
6. qualité effective ;
7. assurance effective ;
8. impact propagé ;
9. éligibilité locale ;
10. éligibilité propagée.

Un recalcul itératif est nécessaire jusqu'au point fixe lorsque le graphe contient des cycles.

21.3. Pseudo-code

```
def propagate(configuration, evidence, context):
    validate_structure(configuration)

    status = initialize_status(configuration)

    for item in configuration.items:
        status[item].direct_evidence = applicable_evidence(
            item=item,
            evidence=evidence,
            context=context,
        )
        status[item].declared_assurance = assurance_from_coverage(
            item=item,
            evidence=status[item].direct_evidence,
        )
        status[item].local_impact = compute_local_impact(
            item=item,
            evidence=status[item].direct_evidence,
            context=context,
        )

    order = dependency_order_or_scc(configuration)

    changed = True
    while changed:
        changed = False

        for item in order:
            dependencies = configuration.dependencies_of(item)

            new_quality = compute_effective_quality(
                item=item,
                dependencies=dependencies,
                status=status,
            )

            new_assurance = compute_effective_assurance(
                item=item,
                dependencies=dependencies,
                status=status,
            )

            new_impact = compute_effective_impact(
                item=item,
                dependencies=dependencies,
```



```

        status=status,
    )

    new_eligibility = compute_eligibility(
        item=item,
        dependencies=dependencies,
        status=status,
        context=context,
    )

    changed |= update_if_different(
        status[item],
        quality=new_quality,
        assurance=new_assurance,
        impact=new_impact,
        eligibility=new_eligibility,
    )

    return build_propagation_report(status)

```

21.4. Cycles

Le graphe de configuration peut contenir certains cycles, notamment via des dépendances mutuelles.

Le moteur DOIT :

- soit rejeter les cycles interdits ;
- soit calculer les composantes fortement connexes ;
- soit appliquer une itération jusqu'au point fixe.

Une implémentation v0.1 PEUT interdire les cycles sur :

```

contains
templates
uses_prompt
uses_model

```

Les cycles autorisés DOIVENT être explicitement déclarés.

21.5. Complexité

Pour un graphe acyclique :

$$O(|V| + |E|)$$

Pour un calcul itératif avec k itérations :

$$O(k(|V| + |E|))$$

Le nombre d'états étant fini et les opérateurs de propagation étant monotones dans un calcul donné, le point fixe doit être atteint.

22. Rapport de propagation

22.1. Structure

```
{
  "report_id": "propagation:01J...",
  "configuration_digest": "sha256:...",
  "context": "baseline_release",
  "computed_at": "2026-07-27T12:30:00Z",
  "valid": false,
  "summary": {
    "eligible": 8,
    "warning": 2,
    "blocked": 1,
    "current": 7,
    "impacted": 3,
    "stale": 1
  },
  "items": [
    {
      "item_ref": {
        "id": "aci:agent:planner",
        "revision_id": "01J..."
      },
      "declared_status": {
        "lifecycle_state": "validated",
        "quality_state": "ok",
        "assurance_state": "assessed"
      },
      "computed_status": {
        "effective_quality": "nok",
        "effective_assurance": "partially_assessed",
        "impact_state": "stale",
        "eligibility_state": "blocked"
      },
      "reasons": [
        {
          "code": "ACM-PROP-001",
          "source_ref": "aci:prompt:planner-system@01J...",
          "relation_type": "uses_prompt",
          "message": "Required prompt quality is NOK."
        }
      ]
    }
  ]
}
```

```
]
}
```

22.2. Reproductibilité

Le rapport DOIT enregistrer :

- la version du moteur ;
- la politique utilisée ;
- le digest de configuration ;
- le digest de l'ensemble de preuves ;
- le contexte de calcul.

23. Invariants normatifs

I1 — Nouvelle révision non validée

Toute nouvelle révision commence en `draft`.

$$newRevision(x) \Rightarrow lifecycle(x) = draft$$

Code :

```
ACM-LIFE-001
```

I2 — Validation liée à une révision exacte

Toute preuve de validation doit référencer un `revision_id` et un digest.

Code :

```
ACM-LIFE-002
```

I3 — Transition valide

Toute transition de lifecycle doit être présente dans la matrice applicable.

Code :

```
ACM-LIFE-003
```

I4 — Validated implique assurance complète

$$lifecycle(x) = validated \Rightarrow effectiveAssurance(x) = assessed$$

Code :

ACM-ASSURANCE-001

I5 — Validated exclut NOK

$$lifecycle(x) = validated \Rightarrow effectiveQuality(x) / nok =$$

Code :

ACM-QUALITY-001

I6 — Archived non réactivable

$$lifecycle(x) = archived \Rightarrow \neg transition(x, s), s / archived =$$

Code :

ACM-LIFE-004

I7 — Release sans blocage

$$baseline.status = released \Rightarrow \forall x \in Required(B), Eligibility(x) = eligible$$

Code :

ACM-BASELINE-002

Une politique de dérogation peut autoriser des warnings, mais jamais des blocages.

I8 — Preuve non transitive par défaut

Une preuve visant x ne couvre pas automatiquement les dépendances ou parents de x .

Code :

ACM-EVIDENCE-001

I9 — Dépendance blocking NOK

Pour toute relation requise et blocking :

$$Q_e(target) = nok \Rightarrow Eligibility(source) = blocked$$

Code :

ACM-PROP-001

I10 — Changement de dépendance et staleness

Si une preuve référence une révision de dépendance différente de la révision courante :

$$dependencySnapshot(e) / currentDependency \Rightarrow applicability(e) = stale$$

Code :

ACM-EVIDENCE-002

I11 — Agent dynamique traçable

Toute instance dynamique doit référencer :

- une factory ;
- un template ;
- un événement de création ;
- une configuration résolue ;
- des permissions résolues.

Code :

ACM-DYNAMIC-001

I12 — Promotion non directe

Une instance `ephemeral` ne peut pas devenir directement un ACI `validated`.

Code :

ACM-DYNAMIC-002

I13 — Non-escalade des permissions

$$P_{instance} \subseteq P_{creator} \cap P_{factory} \cap P_{environment}$$

Code :

ACM-PERM-001

I14 — Calcul déterministe

Pour les mêmes entrées :

$$propagate(C, E, P) = propagate(C, E, P)$$

Code :

ACM-PROP-002

24. Scénarios normatifs de test

24.1. Scénario A — Promotion nominale

Configuration

```
Prompt P1 = validated, ok, assessed
Tool T1 = validated, ok, assessed
Model M1 = validated, ok, assessed
Agent A1 uses P1, T1, M1
Workflow W1 contains A1
```

Résultat attendu

```
A1.effective_quality = ok
A1.effective_assurance = assessed
A1.impact_state = current
A1.eligibility_state = eligible

W1.effective_quality = ok
```

```
W1.effective_assurance = assessed
W1.eligibility_state = eligible
```

Une baseline candidate contenant ces objets peut devenir released si les autres invariants sont satisfaits.

24.2. Scénario B — Dépendance NOK

Changement

```
T1.quality_state = nok
```

Relation :

```
A1 uses_tool T1
required = true
propagation_policy = blocking
```

Résultat attendu

```
T1.effective_quality = nok
A1.quality_state reste inchangé
A1.effective_quality = nok
A1.eligibility_state = blocked
W1.eligibility_state = blocked
baseline release = forbidden
```

Le lifecycle intrinsèque de A1 et W1 ne change pas automatiquement.

24.3. Scénario C — Nouvelle version de prompt

Situation initiale

```
A1 validated with P1 revision R1
Evidence E1 covers A1 + P1@R1
```

Changement

```
P1 revision R2 replaces R1
A1 revision R2 uses P1@R2
```

Résultat attendu

Pour la nouvelle révision de A1 :

```
lifecycle_state = draft
quality_state = unknown
assurance_state = unassessed
```

La preuve E1 reste historiquement valide pour A1@ancienne-révision, mais elle n'est pas applicable à la nouvelle révision.

```
A1.new_revision.impact_state = current
A1.new_revision.effective_assurance = unassessed
A1.new_revision.eligibility_state = blocked
```

Pour une ancienne configuration utilisant encore P1@R1 :

```
impact_state peut devenir impacted
eligibility_state peut devenir warning
```

selon le statut de R1.

24.4. Scénario D — Agent dynamique conforme

Configuration

```
Factory F1 = validated
Template A-template = validated
Prompt P1 = validated
Tool T1 = validated
```

La factory crée une instance avec :

- template inchangé ;
- outil T1 ;
- permissions conformes ;
- spécialisation textuelle autorisée ;
- aucun changement de modèle.

Résultat attendu

```
promotion_state = ephemeral
quality_state = unknown
```



```
effective_assurance = partially_assessed
eligibility_state = warning
```

Après contrôles runtime réussis :

```
quality_state = ok for execution scope
effective_assurance = assessed for execution scope
runtime eligibility = eligible
```

Ces états ne transforment pas automatiquement l'instance en ACI permanent.

24.5. Scénario E — Agent dynamique non autorisé

Une instance est créée :

- sans factory ;
- avec un outil non autorisé ;
- avec une permission supérieure à celle du créateur.

Résultat attendu

```
effective_quality = nok
effective_assurance = unassessed
eligibility_state = blocked
drift classification = undeclared_instance
permission drift = critical
```

Le runtime graph est marqué bloqué pour toute promotion ou réutilisation.

24.6. Scénario F — Dépendance dépréciée

```
Prompt P1.lifecycle_state = deprecated
Agent A1 uses P1
```

Nouvelle baseline

Par défaut :

```
A1.impact_state = impacted
A1.eligibility_state = blocked
```

Une politique de dérogation peut produire :

```
A1.eligibility_state = warning
```

avec justification obligatoire.

Baseline historique

La baseline existante ne change pas automatiquement d'état.

Elle peut recevoir :

```
reassessment_required
```

dans un registre opérationnel distinct.

25. Événements normatifs minimaux

Une implémentation conforme DOIT pouvoir enregistrer :

```
aci.created  
aci.submitted  
aci.rework_requested  
aci.validation_approved  
aci.rejected  
aci.deprecated  
aci.archived  
  
baseline.created  
baseline.released  
baseline.superseded  
baseline.withdrawn  
  
evidence.recorded  
evidence.invalidated  
evidence.expired  
  
impact.detected  
impact.resolved  
  
runtime_object.retained  
runtime_object.promoted  
runtime_object.rejected  
runtime_object.expired
```

Chaque événement DOIT contenir :

- un identifiant unique ;

- une cible exacte ;
- un état précédent ;
- un état suivant lorsque pertinent ;
- un acteur ;
- un timestamp ;
- une raison ;
- des références de preuve lorsque requises.

26. Schéma minimal de politique

Une politique de propagation locale PEUT prendre la forme :

```
{
  "policy_id": "policy:propagation:default-v0.1",
  "version": "0.1.0",
  "quality_order": [
    "ok",
    "unknown",
    "to_improve",
    "nok"
  ],
  "eligibility_order": [
    "eligible",
    "warning",
    "blocked"
  ],
  "impact_order": [
    "current",
    "impacted",
    "stale"
  ],
  "release_rules": {
    "require_validated": true,
    "require_assessed": true,
    "allow_to_improve": true,
    "allow_deprecated": false,
    "allow_warning": false
  },
  "relation_defaults": {
    "uses_prompt": {
      "required": true,
      "propagation_policy": "blocking"
    },
    "uses_tool": {
      "required": true,
      "propagation_policy": "blocking"
    }
  }
}
```

```
}  
}
```

ACM Core v0.1 ne nécessite pas un langage de règles plus général.

27. API logique minimale

Une implémentation conforme devrait exposer des opérations équivalentes à :

```
transition_aci(  
    revision_ref,  
    target_state,  
    actor,  
    evidence_refs,  
) -> LifecycleEvent
```

```
transition_baseline(  
    baseline_ref,  
    target_state,  
    actor,  
) -> BaselineEvent
```

```
evaluate_evidence_applicability(  
    configuration,  
    evidence,  
) -> EvidenceApplicabilityReport
```

```
propagate_status(  
    configuration,  
    evidence,  
    context,  
    policy,  
) -> PropagationReport
```

```
promote_runtime_object(  
    runtime_ref,  
    target_state,  
    actor,  
) -> PromotionEvent
```

28. Critères de conformité

Une implémentation conforme DOIT :

1. supporter les cinq lifecycle states des ACI ;
2. supporter les quatre lifecycle states des baselines ;
3. supporter les runtime states normatifs ;
4. supporter le lifecycle de promotion des objets runtime ;
5. empêcher les transitions interdites ;
6. distinguer qualité et assurance ;
7. calculer impact et éligibilité ;
8. appliquer les politiques de propagation sur les relations ;
9. invalider les preuves devenues non applicables ;
10. produire un rapport déterministe ;
11. conserver les raisons de chaque état calculé ;
12. vérifier les invariants I1 à I14 ;
13. réussir les scénarios A à F.

Une implémentation conforme n'est pas tenue :

- d'exécuter les évaluations LLM ;
- de gérer plusieurs utilisateurs ;
- de posséder une interface graphique ;
- de déployer les baselines ;
- de gérer des graphes distribués ;
- de disposer d'une base de données ;
- d'implémenter une logique probabiliste.

29. Limites connues

29.1. Qualité scalaire

Le modèle réduit la qualité à quatre états globaux.

Il ne représente pas complètement les compromis entre :

- fonctionnalité ;
- sécurité ;
- sûreté ;
- robustesse ;
- coût ;
- latence ;
- explicabilité.

29.2. Agrégation conservatrice

La politique `worst-state` peut produire des blocages trop stricts dans des systèmes comportant de nombreuses dépendances.

Elle est retenue pour sa simplicité et son explicabilité.

29.3. Assurance fondée sur la couverture

La couverture mesure la présence de preuves requises, non leur indépendance, leur qualité méthodologique ou leur fiabilité statistique.

29.4. Contexte limité

Une qualité ou une preuve peut être valide dans un environnement et non dans un autre.

Le modèle repose donc fortement sur le `scope` des preuves.

29.5. Runtime probabiliste

Une instance conforme à une définition validée peut toujours produire un comportement indésirable.

Le modèle ne remplace pas l'observabilité ni les contrôles runtime.

29.6. Agents auto-modifiants

Les changements auto-appliqués par un agent ne sont pas pleinement gouvernés dans v0.1.

Ils doivent être représentés comme mutations runtime et ne peuvent pas modifier directement une baseline released.

30. Extensions envisagées

Une version ultérieure pourra introduire :

- qualité vectorielle ;
- règles de propagation configurables par dimension ;
- scoring de confiance des preuves ;
- réutilisation formelle de preuves ;
- propagation probabiliste ;
- lifecycle des déploiements ;
- waivers et dérogations signées ;
- politiques temporelles ;
- propagation incrémentale ;
- graphes cycliques complexes ;
- intégration avec OpenTelemetry ;
- signatures cryptographiques ;
- attestations SLSA ou in-toto ;
- moteurs de règles externes ;

- relations de responsabilité organisationnelle.

31. Résumé formel

Pour chaque ACI x , ACM distingue :

$$Declared(x) = (L_x, Q_x, A_x)$$

où :

- L_x est le lifecycle déclaré ;
- Q_x est la qualité déclarée ;
- A_x est l'assurance intrinsèque.

Le moteur calcule :

$$Computed(x) = (I_x, E_x, Q_x^{eff}, A_x^{eff})$$

où :

- I_x est l'impact ;
- E_x est l'éligibilité ;
- Q_x^{eff} est la qualité effective ;
- A_x^{eff} est l'assurance effective.

La propagation est :

$$Propagate : (G_C, Evidence, Policy, Context) \rightarrow ComputedStatus$$

Le lifecycle n'est pas propagé :

$$L(parent) / aggregate(\mathbb{E}(children))$$

L'éligibilité est propagée selon les relations :

$$E(parent) = aggregateEligibility(self(parent), relations(parent), dependencies(parent))$$

L'assurance dépend de l'applicabilité des preuves :

$$A_x^{eff}(x) = coverage(ApplicableEvidence(x))$$

Une release est autorisée si :

$$ReleaseAllowed(B) \iff \forall x \in Required(B) : \left\{ L_x = validated \ E_x = eligible \ A_x^{eff} = assessed \ Q_x^{eff} \in \{ok, to_be_assessed\} \right\}$$

32. Conclusion normative

ACM Lifecycle and State Propagation Model v0.1 établit les principes suivants :

1. le lifecycle, la qualité et l'assurance sont des dimensions distinctes ;
2. le lifecycle ne se propage pas hiérarchiquement ;
3. les changements se propagent sous forme d'impact ;
4. les dépendances problématiques se propagent sous forme de restrictions d'éligibilité ;
5. les changements de configuration invalident ou rendent périmées les preuves concernées ;
6. les états effectifs sont calculés sans écraser les états intrinsèques ;
7. les baselines released exigent une configuration exacte, validée et non bloquée ;
8. les objets runtime dynamiques restent éphémères tant qu'ils n'ont pas suivi une procédure explicite de promotion ;
9. toute propagation doit être déterministe, explicable et accompagnée de raisons ;
10. toute décision normative doit être rattachée à une révision exacte et à des preuves identifiables.

Le cœur opérationnel peut être résumé par :

Transition + Impact + EvidenceApplicability + Propagation + Eligibility

Ces cinq opérations constituent la base normative nécessaire à l'implémentation du moteur de lifecycle et de propagation d'ACM Core v0.1.